Deep Deterministic Policy Gradient (DDPG) and Twin-Delayed Deep Deterministic Policy Gradient (TD3) for LunarLander Environment

Adrian S. Kazi
Georgia Institute of Technology
Git Hash: xxxxxxxx

March 3, 2025

1 Introduction

Reinforcement Learning solves complex control tasks, with Deep Deterministic Policy Gradient (DDPG) and Twin-Delayed DDPG (TD3) being key algorithms for continuous action spaces. While DDPG provides a solid foundation, it suffers from instability and overestimation bias. TD3 improves stability and efficiency through twin critics, delayed policy updates, and target smoothing.
We implemented both DDPG and TD3 in the LunarLander environment, where an agent controls a spacecraft aiming for a soft, fuel-efficient landing. The state space includes position, velocity, angle, and leg contact, while actions involve thruster control. Rewards encourage smooth landings and penalize crashes or excessive tilt. By systematically testing different hyperparameters, we analyze how design choices such as policy delay, target noise, and critic structure affect stability, exploration, and learning efficiency, demonstrating why TD3 outperforms DDPG in this setting.

2 Theoretical Foundations

2.1 Policy-Based Methods (post-ANN)

Reinforcement learning is not just about predicting values for actions, it is about making decisions. Instead of estimating how good an action is, we go deeper and directly learn a policy—a function that tells us what to do in each state.
Imagine navigating in complete darkness. A value-based method is like memorizing a map: you estimate which path leads to the best outcome. A policy-based method, however, is like learning an instinct—you act based on experience, adjusting movements dynamically without needing an explicit value for every choice.

Value-Based Methods

In value-based methods, we define a value function that predicts the expected future reward. For example, with the action-value function, our Bellman equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(R + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Value-based methods work well in discrete action spaces, where we don't use stochastic policies. They are efficient in environments where exploration is less critical. A good example is chess, which is well-suited for value-based methods. However, value-based methods struggle in high-dimensional continuous action spaces, such as LunarLander.

Policy-Based Methods

Instead of learning values first and choosing actions later, policy-based methods directly learn the policy $\pi(a \mid s)$, which tells us what action to take in every state. Firstly, we define a policy function $\pi_\theta(a \mid s)$ that outputs an action given a state. Then, we train the policy by maximizing the expected return using gradient ascent. Finally, instead of the Bellman equation, we use the Policy Gradient Theorem to update θ, the parameters of our policy.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\Big[R(\tau) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big] \tag{1}$$

Where: $\nabla_\theta J(\theta)$ is the gradient with respect to θ of function J, which represents the expected total reward obtained from following policy π_θ. θ is a vector of parameters (e.g., neural network weights) that define π_θ. This term tells us how to change the policy parameters θ to increase total rewards. $\mathbb{E}_{\tau \sim \pi_\theta}[...]$ is expectation, meaning we average over multiple possible trajectories. $\tau \sim \pi_\theta$ is a trajectory τ sampled from policy π_θ. Since environments are stochastic, different runs may give different rewards. So, we take an average over many trajectories. Return $R(\tau) = \sum_{t=0}^{T} R(s_t, a_t)$, where $R(\tau)$ is the total reward collected in a trajectory τ. $R(s_t, a_t)$ is the reward received at time t, and T is the total length of

the episode or final timestep. A higher return $R(\tau)$ means better performance, so we want to adjust τ to increase this. $\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$ is the log probability of the policy. $\pi_\theta(a_t \mid s_t)$ is the policy function giving probabilities of taking action a_t in state s_t. $\log \pi_\theta(a_t \mid s_t)$ is the log probability of taking action a_t. $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ is the gradient of the log probability with respect to θ, showing how much the probability of a_t changes with θ. $\sum_{t=0}^{T}$ is a summation over all timesteps since each action contributes to the total reward. If an action a_t leads to high rewards, increase its probability. If an action a_t leads to low rewards, decrease its probability.
To sum up our formula adjusts the policy parameters θ so the agent can make better actions with higher probabilities. It tells us in which direction to update θ to increase expected total rewards.

2.2 Actor-Critic

Actor-Critic methods are assembled from 3 main steps Actor, Critic and Advantage Function, let's understand what each means.

Actor

$$J_\pi = \mathbb{E}_{s,a}[log\, \pi(a \mid s) A(s,a)]$$

Actor decides which action to take in a given state. Actor itself represents the policy $\pi_\phi(a \mid s)$. ϕ is array of weights of the policy π, so we update those weights with some policy optimization method (e.g. policy gradient, PPO or A2C).

Critic

$$V_\psi(s) = \mathbb{E}_{a\sim\pi}[Q_{\theta(s,a)}] = \sum_a \pi(a \mid s) Q_\theta(s,a) \text{ or}$$

$$Q_\theta(s,a) = \mathbb{E}_{s'\sim P, a'\sim\pi}[r + \gamma Q_\theta(s',a') \mid s,a] = \sum_{s'} P(s' \mid s,a) \sum_{a'} \pi(a' \mid s')(r + \gamma Q_\theta(s',a'))$$

Critic evaluates the Actor's decision. It can evaluate with one of 2 estimates. State-value function $V_\psi(s)$ which estimates expected return from state s. Action-value function $Q_\theta(s,a)$ which estimates return from state s and action a. Important things in Critic are parameters ψ and θ, where ψ is an array of weights of Artificial Neural Network responsible for V and θ is exactly the same but for Q. Those two parameters are just weights that are updated by gradient descent using TD errors. Critic is designed to use either the State-value function or the Action-value function, depending on the algorithm.

Advantage Function

$$A(s,a) = Q_\theta(s,a) - V_\psi(s) \text{ if we're using Critic with action-value or}$$

$$A(s,a) = r + \gamma V_\psi(s') - V_\psi(s) \text{ if we're using state-value function in Critic.}$$

As we know $Q_\theta(s,a)$ is action-value and $V_\psi(s)$ is state-value. Advantage function measures how much better or worse action. a is compared to the expected return of state s. This provides an estimate of the relative value of action a, rather than its absolute value. After computing this estimation, we favor action a if $A(s,a) > 0$, meaning it is better than the expected action in state s. Conversely, we avoid action a if $A(s,a) < 0$, indicating it is worse than the expected action in s.

Critic updates with Bellman Equation

$$J_v = \mathbb{E}[(V_\psi(s) - y)^2] \text{ ,where } y = r + \gamma V_\psi(s')$$

Here, we follow traditional Bellman formula, use TD Error and Loss function in following way:

Critic Type	Bellman Update	TD Error	Loss Function
State-Value Critic $V_\psi(s)$	$V_\psi(s) \approx r + \gamma V_\psi(s')$	$\delta = r + \gamma V_\psi(s') - V_\psi(s)$	$(r + \gamma V_\psi(s') - V_\psi(s))^2$
Action-Value Critic $Q_\theta(s,a)$	$Q_\theta(s,a) \approx r + \gamma Q_\theta(s',a')$	$\delta = r + \gamma Q_\theta(s',a') - Q_\theta(s,a)$	$(r + \gamma Q_\theta(s',a') - Q_\theta(s,a))^2$

Intuition

It's worthy to note that Actor formula includes $A(s,a)$ which was updated in Advantage Function that couldn't do that without Critic, and Critic updates itself using the Bellman Equation. We can see that there is an iterative, continuous loop, that updates Actor's policy based on Critic's evolving understanding of the environment. $A(s,a)$ in the Actor's formula shows how much better or worse an action is compared to the average action in a given state. If $A(s,a)$ is high, the action is favorable; if it's low, the action is less favorable.

2.3 Deep Deterministic Policy Gradient (DDPG)

DDPG uses Actor-Critic methodology, but it makes 2 important modifications. AC uses a stochastic policy $\pi_\psi(a \mid s)$, which means that it samples actions probabilistically. DDPG replaces this with the deterministic policy $\mu_\psi(s)$, which directly outputs the best action given a state. Furthermore, DDPG uses an Off-Policy Learning Approach. While

AC learns from recent experiences (on-policy), DDPG stores past experiences in a replay buffer and learns from them (off-policy), which stabilizes learning.

Deterministic Actor instead of Stochastic

In DDPG we completely drop idea of stochasticity, Advantage Function and Critic updates with Bellman Equation. Instead Actor in DDPG updates deterministically, by maximizing Q-values directly:

$$\nabla_\phi J_\pi = \mathbb{E}_s[\nabla_a Q_\theta(s,a) \mid_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)]$$

Here we don't use $A(s,a)$, instead we move policy $\mu(s)$ towards actions that increases Q-values. ∇ means simply the gradient maximization of given function. In other words, above formula means that maximization of policy J_π corresponds to computing expected sum of gradients of Q-values w.r.t. to actions, weighted by the gradient of policy parameters, where policy is μ and policy parameters (or weights of the neural network) is defined by ϕ. Thereby ensuring that policy updates push toward actions that maximize Q-values.

Target Q-Network instead of Bellman & Advantage Function

$$\text{In DDPG we use Target Q-Network: } J_Q = \mathbb{E}[(Q_\theta(s,a) - y)^2] \text{ or } y = r + \gamma Q_{\bar{\theta}}(s', \mu_{\bar{\phi}}(s'))$$

Let's break it down for sake of clarity:

$$\text{Critic Loss Function - } J_Q = (Q_\theta(s,a) - y)^2$$

where $Q_\theta(s,a)$ is Critic's Q-value prediction (or expected sum of rewards), meaning $Q_\theta(s,a)$ = Critic's output given (s,a). $y = r + \gamma Q_{\bar{\theta}}(s', \mu_{\bar{\phi}}(s'))$ where r is immediate reward, γ is discount factor and $Q_{\bar{\theta}}(s', \mu_{\bar{\phi}}(s'))$ is target Critic's Q-value. $Q_{\bar{\theta}}(s', \mu_{\bar{\phi}}(s'))$ is future Q-value prediction from the slowly updated target critic. μ_ϕ is the actor (policy function) in DDPG, s' is the next state and a' is the action the actor selects for s'. All parameters with bar at the top are responsible for slow-moving target networks for stable learning. Parameters without bar are Critic & Actor responsible for fast-learning networks.

Target Q-Network instead of Bellman & Advantage Function

In other words DDPG implements Slow and Fast Networks (Target Networks). Fast networks are given by $Q_\theta(s,a)$, where Critic learns to predict Q-values and $\mu_\phi(s)$, where Actor learns to maximize Q-values. Slow networks (Target Networks) are given by $Q_{\bar{\theta}}(s', \mu_{\bar{\phi}}(s'))$, what provides stable Q-values for critic updates and $\mu_{\bar{\phi}}(s')$, what provides stable actions for computing future rewards. If we update the Critic using its own rapidly changing predictions, training becomes unstable. Using slowly updated target networks provides smoother learning.
Now, let's answer question why $Q_{\bar{\theta}}(s', \mu_{\bar{\phi}}(s'))$ and $\mu_{\bar{\phi}}(s')$ are actually slower. The slowness comes from how the target networks that are updated. Instead of updating directly at every training step like the main networks, they are updated gradually using Polyak averaging:

$$\bar{\theta} \leftarrow \tau\theta + (1-\tau)\bar{\theta} \qquad \bar{\phi} \leftarrow \tau\phi + (1-\tau)\bar{\phi}$$

where τ is small value, e.g. 0.005, meaning the target network moves only a tiny bit toward the updated critic/actor at each step. Eventually this improvement causes that slow target networks are responsible for smooth learning, fast critic/actor networks allow learning to adapt and Polyak averaging makes targets more predictable, reducing Q-value oscillations. So the main innovation is balancing fast learning with slow, stable target updates.

2.4 Twin-Q Twin-Delayed Deep Deterministic Policy Gradient (TD3)

Now, let's move to the final update in our scope. TD3 is built on top of DDPG, but it differs in 2 key ways. Twin Critics for Conservative Q-Learning, where instead of using a single Q-value estimator, TD3 introduces two critics Q_{θ_1} and Q_{θ_2}. It takes the minimum Q-value to prevent overestimation bias. Second improvement is Target Policy Smoothing & Delayed Policy Updates. Unlike DDPG, TD3 delays actor updates and adds noise to target actions to avoid overfitting to critic errors.

$$J_Q = \mathbb{E}[(Q_{\theta_1}(s,a) - y)^2] + \mathbb{E}[(Q_{\theta_2}(s,a) - y)^2] \text{ , where } y = r + \gamma\, \mathbb{E}_{a' \sim \pi}[\min(Q_{\bar{\theta}_1}(s',a'), Q_{\bar{\theta}_2}(s',a'))]$$

Here, J_Q is our Critic Loss Function, which measures the error in the critic's Q-value predictions. That's exactly why we use $(Q_\theta(s,a) - y)^2$, which is the loss equation. Then we simply sum it up over all possible state-action pairs. Again, θ is a vector of weights of the neural network. However, we have two vectors of weights θ_1 and θ_2, representing two independent critic networks. If we look at y, we can notice that it consists of Immediate reward r, and Discounted expected return, using the minimum of two Q-values for stability, given by:

$$\min(Q_{\bar{\theta}_1}(s',a'),\ Q_{\bar{\theta}_2}(s',a'))$$

.

But, why do we use two critics? The reason for using two Q-networks comes from the problem of overestimation bias. If one Q-network overestimates, we still have a second one and pick the lower Q-value. A lower Q-value prevents us from overestimating future rewards and ensures more stable training.

Target Policy Smoothing

TD3 introduces stochasticity into target actions, making the Q-function less sensitive to small errors. Let's compare DDPG and TD3 in that scope:

$$a'_{DDPG} = \pi(s') \qquad\qquad a'_{TD3} = \text{clip}(\pi(s') + \epsilon, a_{min}, a_{max}), \text{ where } \epsilon \sim \mathcal{N}(0, \sigma)$$

This ensures that Target actions are not overly precise, preventing overfitting to critic estimation errors. Furthermore Q-values remain smooth, avoiding sharp spikes in the critic function.

Overfitting to Critic Errors

In DDPG, we take a greedy approach, meaning that the selected action is always the best one based on the policy π. However, in TD3, instead of always picking the single best action a, we add noise, introducing controlled exploration. We still stick to our best action, but in an approximated and stochastic way.

Smoother Q-Function of the Critic

With clipping, we cut out extreme values, preventing large Q-value spikes. The critic is forced to approximate values over a broader action space, leading to better generalization.

TD3 Design Philosophy

If we analyze TD3's design philosophy, we see a clear pattern. The researchers aimed to slow down learning when necessary (delayed updates, conservative Q-learning). But at the same time, they encouraged controlled exploration (target policy smoothing). This balances conservatism and exploration, preventing instability. We can call this philosophy "adventurous conservatism". TD3 is both thoughtful about updates (by limiting Q-value overestimation) while still encouraging gradual, structured exploration. To illustrate this balance, let's imagine three types of people. **1.** A person who stays home all their life, afraid of exploring → This is DDPG without noise. The model gets stuck in local optima and never discovers better strategies. **2.** A person who, without preparation, suddenly decides to travel the world → This is a naive stochastic policy without structure. The model explores too much but lacks direction. **3**. A natural-born explorer who travels constantly but in a structured, well-planned way → This is TD3. It explores but does so gradually, without chaos or sudden moves. TD3 allows us to control this balance between stability and exploration. With delayed updates, conservative Q-learning, and target policy smoothing, we get a system that is reliable and precise but never stops learning.

3 Implementation

In this section we are going to provide brief overview of implementation details.

Actor

We're going to use an ANN assembled from 3 layers: Linear(state_dim, 512) with ReLU activation function, Linear(512, 512) with ReLU activation function, Linear(512, 256) with ReLU activation function and Linear(256, action_dim) with tanh function multiplied by the maximum value of action. $ReLU(x) = \max(0, x)$, prevents vanishing gradient, introduces non-linearity for more efficient learning of complex relationships, and is computationally cheap. $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ is a natural choice for continuous space since it ensures that output is within a valid range. It also helps with smooth control of continuous actions and prevents extreme outputs that might destabilize training.

Critic

Since the Critic network estimates the Q-value function, $Q(s, a) = \mathbb{E}[R \mid s, a]$, it predicts the Q-value (expected reward for taking action a in state s. In the Critic class, we've used the same ANN structure as in the Actor. The only difference is that the last linear layer, instead of returning max_action, returns the Q-value. Hence, it's $Linear(256, 1)$.
In TD3 model we need to remember to add 2nd Critic network. Without overcomplication, we will leave 2nd network same as outlined above.

Hyperparameters

We set up base hyperparameters. In further experiments, we will tweak single hyperparameter leaving all others as in the base set up. Base hyperparameters are given by below table.

Hyperparameter	Value	Hyperparameter	Value	Hyperparameter	Value
Learning Rate (Actor)	$3e-4$	Learning Rate (Critic)	$3e-4$	Replay Buffer Size	$1e6$
Batch Size	128	Discount Factor (γ)	0.99	Max Episodes	$1e3$
Max Steps per Episode	$1e3$	Target Reward	200	Min Memory	$1e4$
Target Policy Clip	0.5	Soft Update Parameter (τ)	$5e-3$	Policy Delay	2
Target Policy Noise	0.3				

Table 1: Base Model Hyperparameters.

4 Testing Twin-Delayed DDPG and DDPG

Let's now run TD3 and DDPG on base model to see the outcomes of both algorithms.

Figure 1: Train Episode Reward

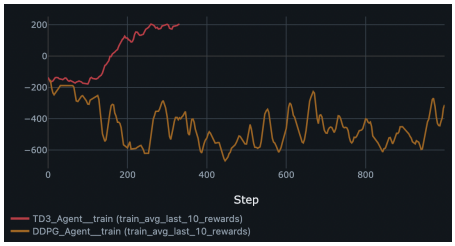

Figure 2: Train Avg. Last 10 Rewards

Figure 3: Test Epsiode Reward

TD3 agent consistently achieves higher and more stable rewards, both during training and testing. It quickly converges to a strong policy, while DDPG remains unstable and struggles to improve. TD3's training rewards steadily increase, whereas DDPG shows high variance and slow learning. Over time, TD3 maintains reliable performance, while DDPG continues fluctuating and fails to reach comparable reward levels.

4.1 Why Twin-Delayed DDPG is better?

TD3 algorithm performs better due to three improvements. Let's go through each improvement, comparing how the problem was handled in TD3 and DDPG.

Q-Function Overestimation

DDPG's Critic is assembled from a single neural network, which serves as a Q-value estimator. This can lead to overestimation of Q-values. The solution is to make the Critic more conservative and slower in terms of updates. TD3 provides a new solution by introducing a second neural network in the Critic and selecting the lower Q-value out of the two networks. This prevents excessive optimism.

$$y_{DDPG} = r + \gamma Q(s', \pi(s')) \qquad y_{TD3} = r + \gamma \min(Q_1(s', a'),\ Q_2(s', a'))$$

The question here is why one network would return a Q-value lower than the other. This occurs due to several reasons. Different function approximation errors, which are introduced randomly, since each neural network has its own errors. Each Q-network uses different gradients due to sampling differences in mini-batches, causing convergence to occur at different speeds. Each Q-network may see a different part of the data distribution. Since they are trained independently, they may be affected by different samples. If one network learns from slightly noisier samples, its estimates will fluctuate more, making it less reliable than the other. If we were to extend this approach, we could introduce a third or fourth Critic network. However, further research is needed to determine whether this is not redundant. For example, in the LunarLander problem, standard TD3 performs well enough.

Too Fast Policy Updates

DDPG updates the Actor (Policy) at every single step using Critic gradients. This means the Actor is updated even though the Critic might not have converged. In TD3, the Actor is updated every second step. While the Critic may still not fully converge in TD3, the likelihood of premature updates is significantly reduced. We could extend this further by updating the Critic every third step to provide even more time for convergence, though this would slow down the training process.

$$\theta^{\pi} \leftarrow \theta^{\pi} + \alpha \nabla_{\theta^{\pi}} Q(s, \pi(s)) \qquad \text{if } t \mod 2 == 0, \quad \theta^{\pi} \leftarrow \theta^{\pi} + \alpha \nabla_{\theta^{\pi}} Q_1(s, \pi(s))$$

Target Policy Smoothing

TD3 introduces stochasticity into target actions, making the Q-function less sensitive to small errors.

$$a'_{DDPG} = \pi(s') \qquad a'_{TD3} = \text{clip}(\pi(s') + \epsilon, a_{\min}, a_{\max}), \quad \text{where } \epsilon \sim \mathcal{N}(0, \sigma)$$

Below, we can see what are the outcomes of two policies.

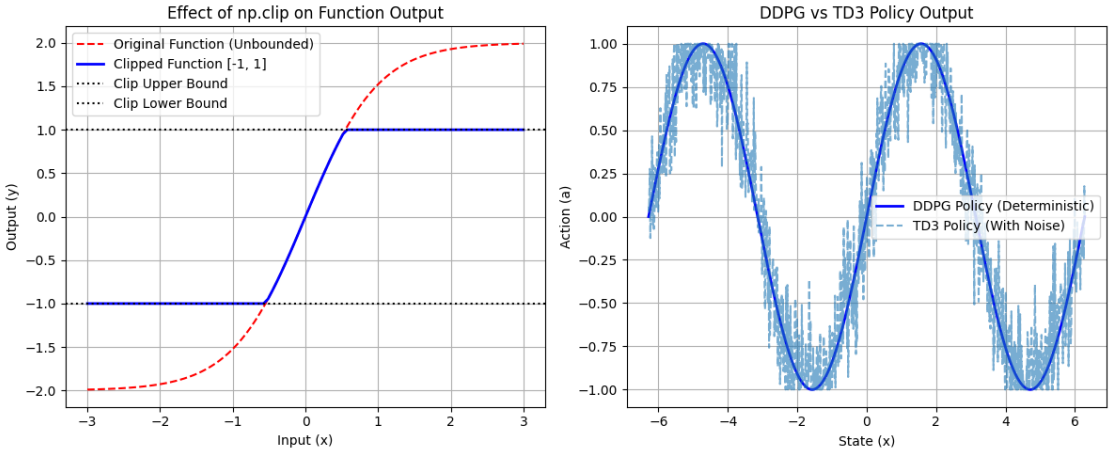

Figure 4: Comparison of TD3 and DDPG Performance

Now, let's go deeper into intuition behind TD3 to understand why TD3 actually have those improvements and why those improvements introduce better overall performance of the model. In DDPG we go with greedy approach, meaning that Action selected will always be the best one based on the Policy π. In TD3, instead picking always the best Action a we add some noise, introducing explorationg. We still stick to our best action but in approximated and stochastic way. With clipping, or cutting out extreme values we prevent from peaks. Critic is forced to approximate values over broader action space. Again, we can notice here some pattern, philosophy that driven researchers in TD3 design. They just minimized chances of model being too fast. On the other hand addition of clipping and noise to the Target Policy researchers introduced more gradual exploration. Obviously researchers tried to make DDPG more conservative, more thoughtful about the updates with limiting it's updates or believes but also they wanted to make Agent more explorative. Therefore, we have - mentioned earlier - the idea of "adventorous conservatism".

5 Hyperparameters Testing

After understanding differences between DDPG and TD3, we can evaluate 3 different hyperparameters and their impact on the optimal policy. Since TD3 is a model that actually solves the problem in a timely manner, let's stick to this model while testing 3 different hyperparameters.

Hyperparameter	**Explanation**	**Formula**
TAU	Controls how fast target networks update. A high value leads to instability, while a low value slows learning.	$\theta' \leftarrow \tau\theta + (1-\tau)\theta'$
POLICY_DELAY	Defines how often the Actor updates. Higher values improve stability but slow adaptation.	if $t \mod d == 0$, $\theta^{\pi} \leftarrow \theta^{\pi} + \alpha_{\pi}\nabla_{\theta^{\pi}} J(\pi)$
TARGET_POLICY_NOISE	Prevents overfitting to Q-values. Excessive noise makes learning chaotic; too little leads to overestimation.	$a' = \text{clip}(\pi'(s') + \epsilon, a_{min}, a_{max})$, where $\epsilon \sim \mathcal{N}(0, \sigma)$

Table 2: Testing hyperparameters

5.1 Metrics

Here we are going to explain the metrics that were used to follow experiments training performance. **Total Rewards** – Sum of rewards collected throughout training. → Indicates overall agent performance. **Average Episode Reward** – Mean reward per episode over time. → Evaluates consistency and learning stability. **Q-Value Estimates** – Predicted reward for state-action pairs. → Helps monitor overestimation bias and critic stability. **Policy Loss** – Measures how much the policy network updates. → Large spikes suggest unstable learning. **Critic Loss** – Measures Q-value prediction errors. → High loss suggests function approximation issues. **Entropy of Actions** – Measures randomness in action selection. → Low entropy means the policy is becoming too deterministic too early. **TD Error (Bellman Residuals)** – Difference between predicted and target Q-values. → Indicates instability in Q-value learning and function approximation errors.

5.2 TAU

From plots, we can analyze the impact of $\tau = 0.01$ (purple) vs 0.005 (yellow) on various metrics.

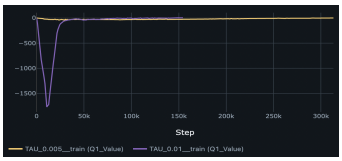

Figure 5: Q1 Value

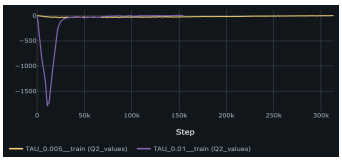

Figure 6: Q2 Value

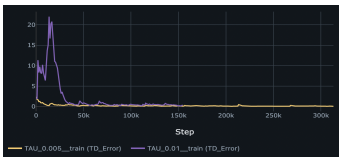

Figure 7: TD Error

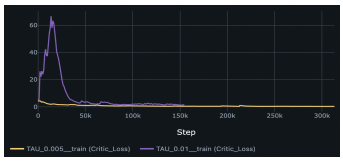

Figure 8: Critic Loss

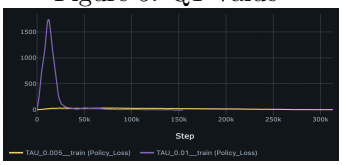

Figure 9: Policy Loss

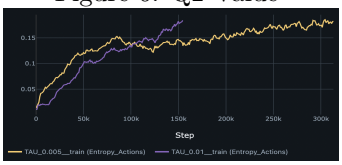

Figure 10: Entropy Actions

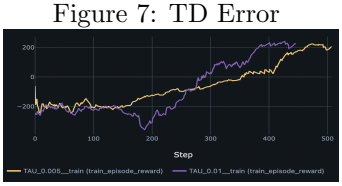

Figure 11: Train Episode

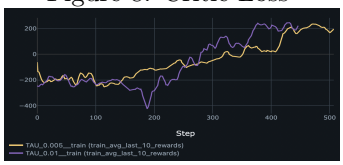

Figure 12: Train Avg. 10 Rewards

Critic Loss - Critic Loss measures how well the Q-function is learning. High Critic Loss means the critic is making large errors in Q-value estimation. Low (but non-zero) indicates stable and efficient learning. A better choice is 0.005 since it stabilizes critic learning, reducing Q-value estimation errors. Higher $\tau = 0.01$ makes the critic unstable early on, requiring more training to settle. **Policy Loss** - Policy Loss represents how much the actor network is being updated. Lower Policy Loss means the policy (Actor) is making smaller, more stable updates. High Policy Loss early on suggests the Actor is making large adjustments, which can lead to instability. $\tau = 0.01$ has an extreme policy loss spike at the beginning before stabilizing. $\tau = 0.005$ has a much lower and more consistent policy loss. Since the Actor depends on Q-values from the critic, an unstable critic causes the Actor to overreact, leading to high loss. Therefore, 0.005 is again the better choice, allowing the Actor to learn more smoothly. **Entropy of Actions** - Entropy of Actions measures how much the agent explores different actions. Higher entropy means more exploration, which is beneficial for early training. A more deterministic policy with lower entropy is good for exploitation. $\tau = 0.005$ results in higher long-term entropy, meaning the agent continues exploring more. $\tau = 0.01$ starts with lower entropy but catches up later. Since $\tau = 0.005$ stabilizes the critic early on, the agent explores actions in a more controlled way. $\tau = 0.01$, due to early instability, starts exploring cautiously but later increases exploration. Once again, $\tau = 0.005$ wins by encouraging better exploration over time. **Q-Values** - Q-values represent the agent's estimate of future rewards for actions. If Q-values fluctuate too much, the agent struggles to learn a consistent reward function. Stable Q-values are necessary for learning a reliable policy. Lower $\tau = 0.005$ keeps Q-values stable, preventing performance collapse. **TD Error** - TD Error measures the difference between the predicted Q-value and the actual reward-based target. A higher TD Error means the critic is making bigger mistakes, while a lower one suggests that the Q-values are converging correctly. Eventually, $\tau = 0.005$ leads to smoother updates and better Q-learning stability. **Training Performance** - Training performance plots show how well the agent is improving its rewards over training. Smoother curves indicate stable learning, while high fluctuations suggest instability. Once again, $\tau = 0.005$ results in faster and more stable reward learning.

5.3 Policy Delay

We've tested 2 Policy Delays, with value of 1 (yellow), and value of 3 (pink).

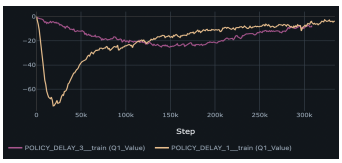

Figure 13: Q1 Value

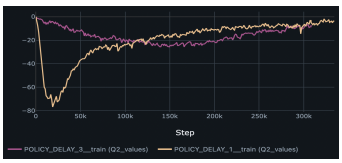

Figure 14: Q2 Value

Figure 15: TD Error

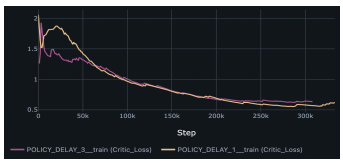

Figure 16: Critic Loss

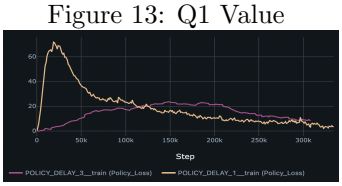

Figure 17: Policy Loss

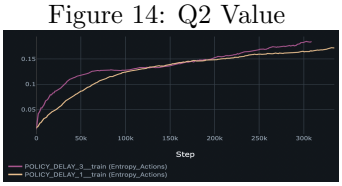

Figure 18: Entropy Actions

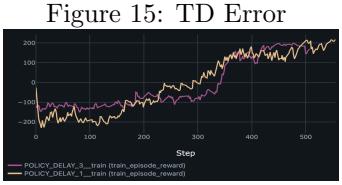

Figure 19: Train Episode

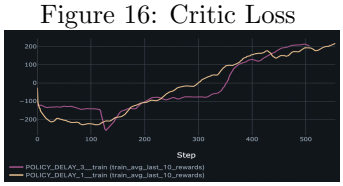

Figure 20: Train Avg. 10 Rewards

Here, we've tested two values of Policy Delays 3 and 1. Of course, value 1 means that there is no delay since we update the Actor at every step, which allows us to investigate why we need Policy Delay. A higher value updates less frequently, allowing the critic to stabilize. A lower value updates more often, which may lead to learning from unstable Q-values.

Critic Loss - Both delays result in similar Critic Loss trends, with slightly more stability for a value of 3. **Policy Loss** - Policy Delay 1 has an extremely high initial policy loss, meaning the actor makes large, unstable updates. A value of 3 results in much lower, more stable Policy Loss. **Entropy of Actions** - Policy Delay 3 maintains slightly higher entropy over time, meaning the agent continues to explore more. A value of 1 has lower entropy early on, suggesting the policy converges faster to a more deterministic strategy. However, Policy Delay 3 encourages sustained exploration, which can be beneficial for long-term learning. A value of 1 leads to quicker exploration, which may be suboptimal if the policy converges prematurely. **Q-values** - Policy Delay 1 initially shows a larger Q-value before recovering. A value of 3 keeps Q-values more stable, leading to smoother learning. This suggests that updating the actor too frequently can cause it to rely on noisy, unstable Q-values, leading to suboptimal updates. **TD Error** - Policy Delay 3 results in a smoother TD Error decay, while Policy Delay 1 has more fluctuations. **Training Performance** - Similarly, in Training Performance, Policy Delay 3 provides smoother and more stable learning. As we can see, Policy Delay makes sense and keeps the Actor learning from an experienced teacher, not from some rookie.

5.4 Target Policy Noise

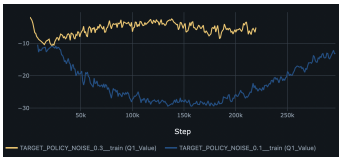

Figure 21: Q1 Value

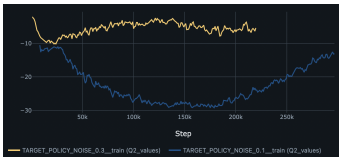

Figure 22: Q2 Value

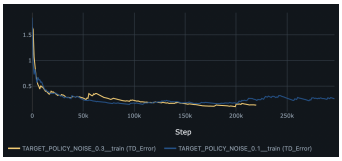

Figure 23: TD Error

Figure 24: Critic Loss

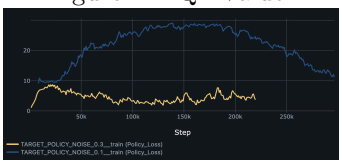

Figure 25: Policy Loss

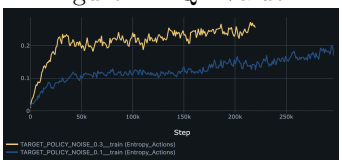

Figure 26: Entropy Actions

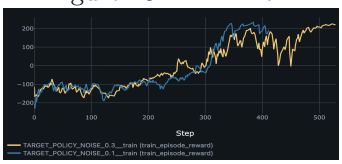

Figure 27: Train Episode

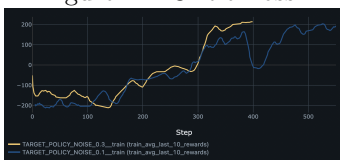

Figure 28: Train Avg. 10 Rewards

Here, we've tested two values of Target Policy Noise 0.3 and 0.1. Higher target policy noise (0.3) adds more randomness to target actions, making the target Q-values smoother and preventing overfitting. Lower noise (0.1) results in more deterministic target actions, potentially leading to overestimation bias. **Critic Loss** - Both noise levels result in decreasing Critic Loss, but noise 0.1 has higher fluctuations towards the end, suggesting it may struggle more with stability in learning. **Policy Loss** - Target Policy Noise 0.1 has a significantly higher Policy Loss early on, indicating larger updates and potential instability. A value of 0.3 keeps Policy Loss lower and more stable over time, meaning updates are more controlled. **Entropy of Actions** - Entropy is consistently higher with noise 0.3, meaning the agent explores more. Lower noise (0.1) results in less exploration, which can lead to faster convergence but may also make the agent more prone to local optima. **Q-values** - Q-values show a clear difference: noise 0.1 results in significantly lower Q-value estimates, suggesting it may be underestimating returns. Noise 0.3 maintains higher, more stable Q-values. **TD Error** - TD Error decays smoothly in both cases, but noise 0.1 has slightly higher variance later in training, indicating potential instability in value estimation. **Training Performance** - Training performance shows that noise 0.3 leads to more stable and consistent learning. Noise 0.1 has a more erratic learning curve, with sharp drops in performance. As we can see, target policy noise plays a crucial role in controlling exploration and preventing overestimation. Noise 0.3 encourages sustained exploration and smoother learning, while noise 0.1 risks early convergence to suboptimal policies.

6 Challenges and Future Improvements

One of the biggest challenges was handling instability in Q-value estimation, especially in DDPG. Overestimation bias led to suboptimal policies, and noisy updates caused fluctuations in learning. TD3 mitigated these issues with twin critics and policy delay, but tuning hyperparameters was still crucial to ensure smooth convergence.
If more time was available, we could explore Soft Actor-Critic (SAC), which introduces entropy regularization to encourage exploration. Unlike TD3, which relies on deterministic policies, SAC learns stochastic policies, making it more robust in highly dynamic environments. However, for this project, TD3 was the better choice because it provides a simpler yet stable framework that aligns well with the LunarLander task, where precision is more critical than broad exploration. SAC's added stochasticity could be beneficial in more complex, high-dimensional environments where exploration plays a bigger role.
Further improvements could also involve adaptive exploration strategies, automatic hyperparameter tuning, and multi-agent setups to push the model's generalization even further.